Ministère de l'Enseignement Supérieur, de la Recherche Scientifique et de la Technologie

Université de Sfax | Faculté des Sciences de Sfax

Département d'Informatique | Master MRSI

QLoRA Fine-Tuning of Gemma-2-2B-IT for Infrastructure as Code Security Remediation

Methodology, Kaggle Execution, and Integration Path

Technical Report — Rapport 7

Prepared by

Ahmedou Yahye Kheyri

Supervised by

Pr. Hala Bezine

Keywords: Fine-Tuning • QLoRA • Gemma-2 • Infrastructure as Code • Security Smells • Kaggle

Academic Year 2025–2026

Abstract

This technical report documents a *first, preliminary* fine-tuning experiment conducted within the broader agentic Infrastructure as Code (IaC) security framework introduced in *Rapport 2*. The experiment is deliberately small in scope: a single run, a small open-weight model ($\approx$ 2 B parameters), a 4,050-record subset of our scraped data, one epoch, and no hyperparameter search. Its purpose is not to deliver a state-of-the-art model but to validate that the training pipeline works end to end and to surface the issues that must be addressed before any scaled experiment is attempted.

We describe (i) the composition of the training data — a 31,748-record scanner-validated dataset obtained through our custom IaC security scraper; (ii) the data preparation pipeline that extracts a 4,050-record *gold subset* used in this first run (roughly 38 % of the available scanner-validated records, sharply filtered for length and non-noop changes); (iii) the Quantized Low-Rank Adaptation (QLoRA) training methodology applied to `google/gemma-2-2b-it` on Kaggle's T4 x2 tier (one of the two available T4 GPUs used by our single-process script), where we observed a training loss of 0.528 and an evaluation loss of 0.467 in 1 h 47 min for one epoch; (iv) a qualitative inspection of the resulting adapter on only three held-out test samples; and (v) the integration path that remains to be executed — adapter merging, GPT-Generated Unified Format (GGUF) conversion, and local Ollama serving.

The report is a standalone technical document but remains tightly coupled to the global thesis project: once merged and served locally, the fine-tuned adapter is intended to serve as the candidate Large Language Model (LLM) for the *Config B* evaluation condition of the experimental plan. At the time of writing, that evaluation has not yet been carried out; only Config A (Checkov alone) has measured results. All Kaggle notebook cells, training hyperparameters, and output logs are reproduced verbatim so that the experiment can be replayed or extended.

Keywords: Fine-Tuning, QLoRA, Gemma-2, Infrastructure as Code, Security Smells, Kaggle, Parameter-Efficient Fine-Tuning.

Contents

Abstract **1**

List of Acronyms **6**

1 Introduction and Positioning **7**

1.1 Context in the Overall Project 7

1.2 Why Fine-Tuning, Why a Small Model 8

1.3 Contributions of This Report 8

1.4 Structure of the Report 9

2 Background **10**

2.1 Low-Rank Adaptation (LoRA) 10

2.2 Quantised LoRA (QLoRA) 11

2.3 Attention Memory and the Context-Length Trade-off 12

2.4 Supervised Fine-Tuning for Code Repair 12

3 Dataset **14**

3.1 Origin: the Scraping Module 14

3.2 Version 1 Statistics 15

3.3 Filtering Pipeline for Supervised Fine-Tuning 15

3.4 Gold Subset: Training-Ready Statistics 16

3.5 Training Sample Format 17

4 Methodology **18**

4.1 Base Model Selection 18

4.2 Parameter-Efficient Fine-Tuning Configuration 18

4.3 Prompt Template 19

4.4 Training Objective 20

4.5 Software Stack 20

5 Execution on Kaggle **21**

5.1 Environment 21

5.2 Cell-by-Cell Walkthrough 21

5.3 Operational Issues Encountered 24

6 Results **25**

6.1 Training Metrics 25

6.2 Qualitative Inspection on Three Test Samples 26

6.3 Adapter Artefacts 27

7 Discussion and Limitations **28**

7.1 Interpretation of Loss Values 28

7.2 Label Noise as the Binding Constraint 28

7.3 Experimental Limitations 29

7.4 Threats to Validity 30

8 Next Steps and Integration Path **31**

8.1 From LoRA Adapter to Deployable Ollama Model 31

8.2 Config B Evaluation 32

8.3 Dataset Quality Improvements 32

8.4 Model Scaling 32

8.5 Long-Term Outlook 33

9 Conclusion **34**

List of Figures

8.1 From adapter to deployable Ollama model. 31

List of Tables

2.1 Base-weight memory for Gemma-2-2B under different precisions, computed from Eq. (2.7). 11

3.1 Dataset v1 overall statistics. 15

3.2 Version 1 breakdown by IaC tool. 15

3.3 Filter pipeline from 31,748 raw records to the gold training subset. . . . 16

3.4 Gold subset distribution after filtering. 16

4.1 Candidate base models for fine-tuning. 18

4.2 QLoRA configuration used for training. 19

6.1 End-of-run training metrics. 25

6.2 Qualitative behaviour on three held-out test samples. 26

List of Acronyms

API Application Programming Interface. 6, 10

CWE Common Weakness Enumeration. 5

GGUF GPT-Generated Unified Format. 1, 6, 27, 30

HF Hugging Face. 14, 17, 20, 27

IaC Infrastructure as Code. 1, 5, 6, 10–12, 30

LLM Large Language Model. 1, 5, 28

LoRA Low-Rank Adaptation. 6–8, 15, 21, 23, 25, 27

NF4 4-bit NormalFloat. 8, 15

PEFT Parameter-Efficient Fine-Tuning. 23

QLoRA Quantized Low-Rank Adaptation. 1, 6–8, 14, 15, 30

RAG Retrieval-Augmented Generation. 5, 29

SFT Supervised Fine-Tuning. 9

VRAM Video Random-Access Memory. 12, 14, 17

Chapter 1

Introduction and Positioning

1.1 Context in the Overall Project

The master's thesis project introduces an *agentic framework* for IaC security analysis and remediation. Seven modules cooperate under a central orchestrator: a contextual analyzer, a knowledge base, a Retrieval-Augmented Generation (RAG) retriever, a fix generator, an external validator (Checkov, GLITCH, Terrascan), a patch formatter, and the orchestrator itself. A detailed architecture and evaluation methodology are given in *Rapport 2*; early experimental results (Config A baseline, Checkov alone) are reported in its Chapter 6.

Among those modules, the **fix generator** is the only component whose quality is bounded by the underlying LLM rather than by deterministic software. In the baseline configuration, the framework can call any OpenAI-, Anthropic-, or Ollama-compatible backend. For local deployment and reproducibility we rely on Ollama serving small open-weight models such as Gemma-3-4B and Gemma-4-e2B. Informal tests on this baseline suggested two points of concern that motivate the present experiment:

- **Generic behaviour**: the off-the-shelf LLM produces plausible fixes but, in our limited manual checks, did not always target the specific Common Weakness Enumeration (CWE) category reported by the analyzer. A systematic measurement remains to be done.
- **Latency**: on a single laptop CPU we observed roughly 320 seconds per fix with the Gemma-4-e2B model, which would make a full dataset evaluation with self-consistency sampling impractical on the current hardware.

A plausible direction is to fine-tune a small open-weight model on the data we already scraped. This report documents a *first, exploratory* attempt in that direction, carried out on Kaggle's free GPU tier.

1.2 Why Fine-Tuning, Why a Small Model

Fine-tuning adapts a pretrained model to a narrow downstream distribution. Compared to in-context prompting alone, supervised fine-tuning on curated (insecure, fixed) pairs is *expected* to offer two advantages: the model may learn domain vocabulary that is sparse in general pretraining corpora (scanner rule identifiers, provider-specific HCL patterns, Kubernetes security-context fields), and the instruction-following behaviour may become more consistent in output format (a compilable IaC block rather than a discursive explanation). Whether these advantages actually materialise for our setup is an empirical question that this first run cannot yet answer.

A small model ($\leq$ 3 B parameters) is chosen deliberately for this first experiment. The agentic framework targets *local* inference as part of security audits that cannot route code through a third-party Application Programming Interface (API). Gemma-2-2B-IT is small enough to fit on an NVIDIA T4 (16 GB) with 4-bit quantisation, can in principle be merged into a standalone GGUF file of roughly 1.5 GB, and is expected to run at usable latency inside Ollama on a commodity laptop. A 2 B model is modest by current standards and is chosen for tractability rather than for expected peak quality; larger candidates (Qwen2.5-Coder-3B, Gemma-2-9B) are deferred to a subsequent iteration.

1.3 Contributions of This Report

The contributions of this report are modest and should be read as a first iteration:

1. A reproducible three-script training pipeline (`01_prepare_dataset.py`, `02_train_qlora.py`, `03_inference.py`) under `Code/training/trainning/`.
2. One QLoRA fine-tuning run on `google/gemma-2-2b-it`, executed on Kaggle's free T4 x2 tier under the constraints imposed by that environment (single-process script using one of the two T4 devices, 9-hour session limit, gated model access).
3. The observed training metrics from that single run (loss curves, runtime, adapter size) and a qualitative look at only three held-out test samples, with a candid discussion of the label noise we encountered.
4. A proposed integration path — not yet executed — that would turn the trained Low-Rank Adaptation (LoRA) adapter into a deployable Ollama model usable by the framework's existing `FixGenerator` class without source-code changes.

Nothing in this report should be read as a quantitative comparison against baselines, and the model produced here has not yet been evaluated inside the full agentic pipeline.

1.4 Structure of the Report

Chapter 2 provides the necessary background on LoRA and QLoRA. Chapter 3 describes the dataset used for training, its origin, statistics, and filtering pipeline. Chapter 4 presents the fine-tuning methodology (model choice, hyperparameters, prompt format). Chapter 5 reproduces the Kaggle execution in full, cell by cell, with captured output. Chapter 6 analyses training metrics and the qualitative inference test. Chapter 7 discusses limitations, particularly dataset label noise, and Chapter 8 outlines the remaining integration steps required to plug the fine-tuned model into the full agentic pipeline.

Chapter 2

Background

2.1 Low-Rank Adaptation (LoRA)

LoRA (Hu *et al.*, 2021)[1] is a parameter-efficient fine-tuning method. Given a frozen weight matrix $W_0 \in \mathbb{R}^{d\times k}$ of a pretrained layer, LoRA augments it with a learned low-rank update $\Delta W = BA$ where

$$B \in \mathbb{R}^{d\times r}, \qquad A \in \mathbb{R}^{r\times k}, \qquad r \ll \min(d,k). \tag{2.1}$$

The forward pass of an adapted layer, applied to an input $\boldsymbol{x} \in \mathbb{R}^k$, is then

$$\boldsymbol{h} \;=\; W_0\,\boldsymbol{x} \;+\; \Delta W\,\boldsymbol{x} \;=\; W_0\,\boldsymbol{x} \;+\; \frac{\alpha}{r}\,B\,A\,\boldsymbol{x}, \tag{2.2}$$

where α is a fixed scaling factor that decouples the learning rate from the rank. Following standard practice, A is initialised from a Gaussian $A_{ij} \sim \mathcal{N}(0,\sigma^2)$ and B from zero, so that $\Delta W = 0$ at step 0 and the adapted model starts identical to the pretrained one.

Trainable parameter count. Let $\mathcal{L}$ be the set of adapted layers and (d_ℓ, k_ℓ) the shape of W_0 at layer ℓ. Only A and B receive gradients, so

$$|\theta_{\text{LoRA}}| \;=\; r\sum_{\ell\in\mathcal{L}}\Big(d_\ell + k_\ell\Big) \;\ll\; \sum_{\ell\in\mathcal{L}} d_\ell\,k_\ell \;=\; |\theta_{\text{full}}|. \tag{2.3}$$

Eq. (2.3) is what makes LoRA attractive: the number of trainable parameters grows linearly in r, not quadratically in the hidden dimension.

Numerical instance (Gemma-2-2B). Our run adapts the four attention projections of every transformer block. With a hidden size $d_{\text{model}} = 2304$, an attention projection size d_{attn}, and $L = 26$ blocks, applying LoRA at rank $r = 16$ to $\{W_q, W_k, W_v, W_o\}$ yields

$$|\theta_{\text{LoRA}}| \;\approx\; r\cdot L\cdot \sum_{W\in\{q,k,v,o\}}\Big(d_{\text{in}}(W) + d_{\text{out}}(W)\Big) \;=\; 9{,}273{,}344, \tag{2.4}$$

[1] E. Hu *et al.*, *LoRA: Low-Rank Adaptation of Large Language Models*, arXiv:2106.09685.

i.e. ≈ 9.3 M trainable parameters, which is a fraction $9.3 \times 10^6 / 2.6 \times 10^9 \approx 0.35\,\%$ of the full base model. The resulting adapter file is 25 MB on disk instead of ≈ 5 GB for the full fp16 model — a $\sim$200$\times$ reduction that is the property exploited by QLoRA.

2.2 Quantised LoRA (QLoRA)

QLoRA (Dettmers *et al.*, 2023)[2] combines LoRA with 4-bit quantisation of the *base* weights. Conceptually, each base weight tensor W_0 is replaced by its 4-bit NormalFloat (NF4) quantised representation

$$Q_{W_0} \;=\; \mathrm{quant}_{\mathrm{NF4}}(W_0; s), \qquad \widetilde{W}_0 \;=\; \mathrm{dequant}_{\mathrm{NF4}}(Q_{W_0}; s), \tag{2.5}$$

where s is a per-block scaling constant and $\widetilde{W}_0 \approx W_0$. The adapted forward pass of Eq. (2.2) is therefore evaluated with the dequantised weights while the low-rank update stays in 16-bit precision:

$$\boldsymbol{h} \;=\; \widetilde{W}_0\,\boldsymbol{x} \;+\; \frac{\alpha}{r}\,B\,A\,\boldsymbol{x}. \tag{2.6}$$

Gradients flow exclusively through (A, B); the quantised Q_{W_0} is never updated. Double quantisation compresses the scaling constants themselves, and NF4 uses a bit-allocation optimised for the (approximately normal) distribution of pretrained weights. These details are inherited from Dettmers *et al.* (2023) and implemented by `bitsandbytes`.

Memory footprint. Storing a parameter tensor of size P in a representation that uses b bits per entry costs

$$M(P, b) \;=\; \frac{b}{8}\,P \quad \text{bytes}. \tag{2.7}$$

For a $P \approx 2.6 \times 10^9$ model, Eq. (2.7) gives the base-weight memory cost shown in Table 2.1. The NF4 row includes the double-quantised scale overhead (~ 0.5 bits/parameter), hence the ≈ 1.6 GB value rather than the naive 1.3 GB.

Table 2.1: Base-weight memory for Gemma-2-2B under different precisions, computed from Eq. (2.7).

Representation	Bits/param	Formula	Memory
fp32	32	$4P$	≈ 10.4 GB
fp16 / bf16	16	$2P$	≈ 5.2 GB
NF4 (w/ double-quant)	4+0.5	$(b/8)\,P$	≈ 1.6 GB

Combining Table 2.1 with the adapter cost of Eq. (2.4) (≈ 9.3 M parameters in fp16,

[2] T. Dettmers *et al.*, *QLoRA: Efficient Finetuning of Quantized LLMs*, arXiv:2305.14314.

$\approx$ 18 MB) and 8-bit AdamW states (Dettmers, 2022)[3] for those same parameters, the static part of the Video Random-Access Memory (VRAM) budget stays well below 2 GB. The bulk of the remaining 14 GB on a T4 is therefore available for activations (which grow with the sequence length, see Section 2.3) and for gradient-checkpointing recomputation buffers. This is precisely what allows QLoRA to run on Kaggle's free T4 tier at sequence length 2,048.

2.3 Attention Memory and the Context-Length Trade-off

The activation cost is dominated by self-attention. For one attention layer processing a batch of B sequences of length n with H heads, the attention logits $QK^\top$ alone occupy

$$M_{\text{attn}}(B, H, n, b) \;=\; \frac{b}{8}\, B\, H\, n^2 \quad \text{bytes}, \tag{2.8}$$

so the memory grows *quadratically* in n. Over all L transformer blocks the dominant term scales as $\mathcal{O}\big(L\, B\, H\, n^2\big)$. In our configuration ($L = 26$, $H = 8$, $B = 1$, fp16 compute, $n = 2{,}048$), the naive per-layer logits cost is already $M_{\text{attn}} = 2 \cdot 1 \cdot 8 \cdot 2048^2 \approx 67$ MB, which is why doubling the context to $n = 4{,}096$ quadruples this term and quickly exhausts the ~ 14 GB left on a T4 after Table 2.1. This quadratic scaling is the immediate reason for capping the combined prompt+completion length during data preparation (Section 3.3).

2.4 Supervised Fine-Tuning for Code Repair

Our task is a form of Supervised Fine-Tuning (SFT) (Ouyang *et al.*, 2022)[4] where the model receives an instruction prompt and produces a target completion. The instruction describes the detected security smells; the completion is the human-written fixed version of the script.

Training objective. A training example is a pair $(\boldsymbol{x}, \boldsymbol{y})$ with $\boldsymbol{x} = (x_1, \ldots, x_m)$ the tokenised prompt and $\boldsymbol{y} = (y_1, \ldots, y_n)$ the tokenised target completion. Writing θ for the learnable parameters (the LoRA factors A and B of Eq. (2.1); recall that $\widetilde{W}_0$ is

[3] T. Dettmers, *8-bit Optimizers via Block-wise Quantization*, ICLR 2022.

[4] L. Ouyang *et al.*, *Training Language Models to Follow Instructions with Human Feedback*, NeurIPS 2022.

frozen), the causal language-modelling loss used in our run is

$$\mathcal{L}(\theta) \;=\; -\sum_{t=1}^{n} m_t \,\log p_\theta\Big(y_t \mid \boldsymbol{x},\, y_{<t}\Big), \tag{2.9}$$

where $y_{<t} = (y_1, \ldots, y_{t-1})$ and $m_t \in \{0, 1\}$ is a *completion-only* mask that equals 1 on target tokens and 0 on prompt tokens. This mask is important: it prevents the model from being rewarded for reproducing the insecure `code_before` that sits inside the prompt, and it concentrates the gradient signal on the corrective edit.

Gradient update. Because gradients flow only through the LoRA adapter (Eq. (2.6)), at each optimisation step k only (A, B) are touched:

$$(A_{k+1}, B_{k+1}) \;=\; (A_k, B_k) - \eta\, \nabla_{(A,B)}\, \widehat{\mathcal{L}}(\theta_k), \tag{2.10}$$

with η the learning rate and $\widehat{\mathcal{L}}$ the mini-batch estimate of Eq. (2.9) averaged over $B_{\text{eff}} = B_{\text{micro}} \times G$ examples, where B_{micro} is the per-device micro-batch size and G the number of gradient-accumulation steps. In our configuration $B_{\text{micro}} = 1$ and $G = 8$, giving an effective batch size $B_{\text{eff}} = 8$.

Framing. Unlike classical code generation, our setting is closer to *code-to-code translation*: the model sees broken code and must emit valid code that preserves functional equivalence while removing the security smell. This narrow, constrained task is well suited to parameter-efficient fine-tuning with limited data.

Chapter 3

Dataset

3.1 Origin: the Scraping Module

The training data comes from a custom scraper we developed and documented in `Code/scraping/`. The scraper combines five complementary strategies to collect (insecure, fixed) IaC pairs from public sources:

1. **GitHub Commit Search** with date-window partitioning to bypass the 1,000-result hard cap. Each commit whose message mentions security-related keywords is inspected; for every IaC file changed in the commit, the version before and after the fix is downloaded through the Core API.

2. **GHArchive discovery**: hourly dumps of all public GitHub events are pre-filtered by repository-name regex before resolving push events, which bypasses the Search API quota entirely.

3. **Scanner test suites**: Checkov, KICS, and tfsec all ship with paired "failed" and "passed" example files. These are a natural source of clean before/after pairs.

4. **Known vulnerable repositories**: TerraGoat, Kubernetes-Goat, and OWASP WrongSecrets contribute intentionally insecure files.

5. **Scanner-validated labels**: a post-processing pass runs Checkov, tfsec, and KICS on every record and attaches `validated_smells_before` / `validated_smells_after` lists. This produces *scanner-confirmed* ground-truth labels suitable for supervised training.

The scraper was run for several days with progress checkpointing, SHA-256 deduplication, and exponential-backoff retries to cope with GitHub rate limits. Full implementation details are in `Code/scraping/README.md`.

3.2 Version 1 Statistics

The frozen *version 1* dataset (`scraping/output/dataset_v1_validated.jsonl`, 322 MB) contains 31,748 records after global deduplication. Table 3.1 summarises its composition.

Table 3.1: Dataset v1 overall statistics.

Property	Value
Total records (deduplicated)	31,748
Records with fix (`has_fix`=True)	16,362 (51.5 %)
Scanner-validated records	10,791 (34.0 %)
Source repositories	> 2,000
Security smell types	18
Pre-existing train / val / test split	25,383 / 3,189 / 3,176
Average combined code length	5,760 characters
Maximum combined code length	263,965 characters

Table 3.2: Version 1 breakdown by IaC tool.

IaC tool	Records	Share
Terraform	15,863	50.0 %
Kubernetes	8,919	28.1 %
Ansible	4,161	13.1 %
Docker	2,333	7.3 %
CloudFormation	472	1.5 %

The smell distribution is heavily skewed toward Docker- and Kubernetes-centric issues: `missing_resource_limits` (4,505), `unpinned_base_image` (3,429), `overly_permissive_cidr` (1,614), `latest_image_tag` (1,009), and `capabilities_added` (884) account for most of the data. Credential-related smells are under-represented (< 500 records each) because commits that fix hard-coded secrets are often held in private repositories or heavily redacted.

3.3 Filtering Pipeline for Supervised Fine-Tuning

Raw records are not immediately usable for training. The preparation script `01_prepare_dataset.py` applies six filters in sequence (Table 3.3).

Table 3.3: Filter pipeline from 31,748 raw records to the gold training subset.

Filter	Remaining	Kept
Input (all validated records)	31,748	100.0 %
`has_fix` = `True`	16,362	51.5 %
Scanner-validated (`gold_only`=True)	∼10,791	34.0 %
Non-empty `code_before` and `code_after`	∼10,500	—
Combined length < 8,000 chars	∼5,500	—
Non-noop fix (`before` ≠ `after`)	4,050	12.76 %
Pre-existing train split (80 %)	**3,227**	—

The combined length cap is a direct consequence of the VRAM budget: per Eq. (2.8), the attention memory grows as $\mathcal{O}(n^2)$ in the sequence length, so at $n = 2{,}048$ tokens the activations fit on a T4 alongside the model state but a longer context would not. The scanner-validated filter is the most aggressive (it discards roughly two thirds of fix pairs); it ensures that the `code_before` is flagged as vulnerable by at least one static analyser, rather than being selected only from keywords in the commit message. This filter acts only at the *record* level and does not guarantee that the recorded diff addresses the same vulnerability — a limitation we return to in Chapter 7.

The 4,050-record gold subset used in this experiment is, by design, a small fraction of what is available. The same pipeline can produce a larger (but noisier) corpus of up to ≈ 10,791 scanner-validated records if the length and non-noop filters are relaxed; exploring that regime is deferred to a later iteration.

3.4 Gold Subset: Training-Ready Statistics

After filtering, the resulting corpus counts 4,050 records (3,227 train, 411 validation, 412 test). Table 3.4 shows a much more balanced distribution by IaC tool than the raw dataset.

Table 3.4: Gold subset distribution after filtering.

Tool	Records	Share
Docker	1,300	32.1 %
Terraform	1,229	30.3 %
Kubernetes	1,197	29.6 %
CloudFormation	203	5.0 %
Ansible	121	3.0 %

3.5 Training Sample Format

Each sample is a JSON object with the slim schema shown in Listing 3.1. The training script reads these records and renders them through the tokenizer's chat template; the full instruction prompt is reconstructed on the fly (Chapter 4).

```
{
  "id": "a1b2c3d4",
  "iac_tool": "terraform",
  "smell_types": ["hardcoded_credential", "
   overly_permissive_cidr"],
  "cwes": ["CWE-798", "CWE-732"],
  "code_before": "resource aws_s3_bucket ...",
  "code_after":  "resource aws_s3_bucket ...",
  "split": "train"
}
```

Listing 3.1: Schema of a training sample.

Chapter 4

Methodology

4.1 Base Model Selection

Four candidate base models were considered (Table 4.1). The choice trades off quality, licensing, hardware constraints, and coherence with the rest of the framework.

Table 4.1: Candidate base models for fine-tuning.

Model	Size	Gated	Remark
google/codegemma-2b-it	2 B	Yes	Code-specialised
google/gemma-2-2b-it	2 B	Yes	**Chosen** — matches local Ollama stack
Qwen/Qwen2.5-Coder-3B-Instruct	3 B	No	Strong code model, non-gated
bigcode/starcoder2-3b	3 B	No	No chat template

We selected `google/gemma-2-2b-it` for three pragmatic reasons, none of which imply it is the best candidate: (i) the same Gemma family is already served locally through Ollama (Gemma-3-4B and Gemma-4-e2B), so integration does not introduce a new architecture; (ii) the instruction-tuned variant follows a published chat template that maps onto our instruction/response pair; (iii) at 2 B parameters it is the smallest Gemma-2 variant, leaving VRAM headroom on a T4 and keeping the experiment within a few hours. A systematic model comparison is out of scope for this first run.

The gated-repository restriction on Gemma-2 required manual license acknowledgement on the Hugging Face (HF) model page and a HF access token exposed to Kaggle through a notebook secret.

4.2 Parameter-Efficient Fine-Tuning Configuration

Table 4.2 lists the exact QLoRA configuration. Values were chosen conservatively from published recipes and were not tuned by search — a deliberate decision, because the first goal is to demonstrate a working baseline, not to optimise hyperparameters.

Table 4.2: QLoRA configuration used for training.

Parameter	Value
Base model	`google/gemma-2-2b-it`
Quantisation	4-bit NF4 (double-quant, fp16 compute)
LoRA rank r	16
LoRA α	32
LoRA dropout	0.05
Target modules	`q_proj, k_proj, v_proj, o_proj`
Trainable parameters	9,273,344 (0.35 % of 2.6 B)
Learning rate	2×10^{-4}
Scheduler	Cosine
Warmup ratio	3 %
Effective batch size	8 (1 × 8 grad. accumulation)
Epochs	1
Optimiser	Paged AdamW 8-bit
Max sequence length	2,048 tokens
Precision	fp16 (bf16 unavailable on T4)
Gradient checkpointing	Enabled (non-reentrant)
Random seed	42

4.3 Prompt Template

During training we apply Gemma-2's native chat template to a two-turn conversation: a user turn describing the task and a model turn containing the ground-truth fixed code. The template is reproduced in Listing 4.1.

```
You are a security expert specializing in Infrastructure as
   Code (IaC).
Fix the following {iac_tool} script. Detected smells: {smells}.
Return ONLY the corrected script, no explanation.

‘‘‘
{code_before}
‘‘‘
```

Listing 4.1: Instruction template applied to every training sample.

Placeholders are filled per sample: `{iac_tool}` with one of `terraform`, `kubernetes`, `docker`, `ansible`, `cloudformation`; `{smells}` with a comma-separated list of smell types; `{code_before}` with the insecure code. The target completion is the `code_after`

field.

4.4 Training Objective

We use the SFT loss introduced in Eq. (2.9): a token-level causal cross-entropy computed only over the model turn (the fixed code), with $m_t = 0$ on every token that belongs to the user turn. Parameters are updated by the rule of Eq. (2.10) with $\eta = 2 \times 10^{-4}$ and effective batch size $B_{\text{eff}} = 8$.

4.5 Software Stack

All training code is Python 3.12. The key library versions are pinned in `training/trainning/requirem` `transformers 4.44.2`, `datasets 2.21.0`, `peft 0.12.0`, `accelerate 0.34.2`, `trl 0.10.1`. A single exception is `bitsandbytes`, which was upgraded in place to version 0.49.2 on Kaggle because the pinned 0.43.3 lacks CUDA 12.8 binaries, which is the CUDA version of the current Kaggle T4 image.

Chapter 5

Execution on Kaggle

5.1 Environment

The experiment was executed on Kaggle's free GPU tier with the following accelerator and runtime settings:

- Accelerator: Kaggle *T4 x2* tier — two NVIDIA T4 devices (16 GB VRAM each). Our training script is single-process and therefore uses only one of the two T4s; the second device is available but unused in this run.
- CUDA version: 12.8.
- Python: 3.12.
- Session limit: 9 hours per notebook.
- Internet: enabled (required to fetch base model weights).
- Dataset attachments: two custom Kaggle datasets, one for the filtered training corpus and one for the Python training scripts.

The Gemma-2-2B-IT license must be manually acknowledged on the HF model page before any model download will succeed. The HF token is exposed to the notebook through Kaggle's encrypted secret store under the name `HF_TOKEN`.

5.2 Cell-by-Cell Walkthrough

The notebook contains five cells. Each cell is reproduced below together with the captured output as it actually appeared during the run.

Cell 1 — Install dependencies

```
!pip install -q -U transformers==4.44.2 datasets==2.21.0 peft==0.12.0 \
    accelerate==0.34.2 bitsandbytes==0.43.3 trl==0.10.1
```

On the first session, this cell completes with warnings about unrelated Kaggle-pre-installed packages (`bigframes`, `tpot`, `s3fs`) whose version constraints are incompatible with the pins above. These warnings are harmless because none of those libraries are imported by the training script.

A subsequent independent upgrade is required because `bitsandbytes 0.43.3` does not ship a binary for CUDA 12.8 (Kaggle's current default):

```
!pip install -q -U bitsandbytes
```

```
bitsandbytes version: 0.49.2
CUDA available: True | CUDA version: 12.8
```

Listing 5.1: Post-upgrade verification output.

Cell 2 — Expose HF token

```
import os
from kaggle_secrets import UserSecretsClient
os.environ["HF_TOKEN"] = UserSecretsClient().get_secret("
    HF_TOKEN")
print("HF_TOKEN set:", os.environ["HF_TOKEN"][:8] + "...")
```

```
HF_TOKEN set: hf_xxxxx...
```

Any subsequent kernel restart clears this environment variable; Cell 2 must be re-executed before Cells 3–5 to avoid `401 Unauthorized` errors when fetching gated model weights.

Cell 3 — Smoke test (200 training samples, ∼ 15 min)

```
!python /kaggle/input/datasets/.../qlora-scripts/02
    _train_qlora.py \
     --model google/gemma-2-2b-it \
     --data-dir /kaggle/input/datasets/.../iac-security-v1/
    data \
     --output-dir /kaggle/working/lora_smoke \
     --epochs 1 --max-train-samples 200
```

The smoke test is used as a sanity check: it exercises the data path, chat-template rendering, and gradient flow before committing to the longer run. Expected markers are a `trainable params` line around 9 M and a non-diverging loss over the 25 optimisation steps. The smoke test produces a single final loss value rather than a loss curve; a decreasing curve across steps is only observable in the full run.

```
[info] starting training...
{'loss': 0.745, 'grad_norm': 0.332, 'learning_rate': 0.0, 'epoch':
    1.0}
{'train_runtime': 429.72, 'train_samples_per_second': 0.465,
 'train_steps_per_second': 0.058, 'train_loss': 0.745, 'epoch': 1.0}
100%|..........| 25/25 [07:09<00:00, 17.19s/it]
[done] adapter saved to /kaggle/working/lora_smoke
```

Listing 5.2: Smoke-test tail.

The recorded step rate of 17.2 s/step allows us to extrapolate the full run: 3,227 / 8 = 403 optimisation steps ≈ 2 h, safely below Kaggle's 9-hour session ceiling.

Cell 4 — Full training (1 epoch on the gold subset)

```
!python /kaggle/input/datasets/.../qlora-scripts/02
    _train_qlora.py \
     --model google/gemma-2-2b-it \
     --data-dir /kaggle/input/datasets/.../iac-security-v1/
    data \
     --output-dir /kaggle/working/lora_out \
     --epochs 1 --batch-size 1 --grad-accum 8 --lr 2e-4
```

```
{'eval_loss': 0.46743, 'eval_runtime': 217.61,
 'eval_samples_per_second': 1.889,
 'eval_steps_per_second': 1.889, 'epoch': 0.99}
{'train_runtime': 6425.84, 'train_samples_per_second': 0.502,
 'train_steps_per_second': 0.063, 'train_loss': 0.528, 'epoch': 1.0}
100%|..........| 403/403 [1:47:05<00:00, 15.94s/it]
[done] adapter saved to /kaggle/working/lora_out
```

Listing 5.3: Full-run summary.

Cell 5 — Inference check on held-out test samples

```
!python /kaggle/input/datasets/.../qlora-scripts/03
    _inference.py \
     --model google/gemma-2-2b-it \
     --adapter /kaggle/working/lora_out \
     --input-file /kaggle/input/datasets/.../iac-security-v1
    /data/test.jsonl \
     --n-samples 3
```

Cell 5 reloads the 4-bit base model, attaches the saved LoRA adapter through the

`peft` library, builds the same instruction prompt used during training, and generates up to 1,024 new tokens per sample at a temperature of 0.2. Three test records are processed and printed in a `(input / ground truth / model output)` format. The output is examined in Chapter 6.

5.3 Operational Issues Encountered

Three issues appeared during the run and are documented here for reproducibility:

1. **403 Forbidden on Gemma-2**: the base model is gated. Resolution: visit the HF model page, click "Acknowledge license", wait for the instant approval, re-run.
2. **Bitsandbytes CUDA mismatch**: the pinned `bitsandbytes==0.43.3` fails with `No module named 'triton.ops'` on CUDA 12.8. Resolution: upgrade to 0.49.2 with `!pip install -q -U bitsandbytes`.
3. **Kernel restart wipes env vars**: after any idle timeout, `HF_TOKEN` must be re-exported through Cell 2 before any subsequent subprocess call can authenticate.

Chapter 6

Results

6.1 Training Metrics

The single full run completed in 1 h 47 min over 403 optimisation steps, close to the pre-run extrapolation based on the smoke-test step time. The metrics below come from that one run with a fixed random seed and should be read as a single observation, not an estimate with a confidence interval.

Table 6.1: End-of-run training metrics.

Metric	**Value**
Optimisation steps	403
Wall-clock training time	6,426 s ($\approx$ 1 h 47 min)
Wall-clock evaluation time	218 s
Average step time	15.94 s
Training throughput	0.502 samples/s
Final training loss	0.528
Final evaluation loss	0.467
Adapter size (`safetensors`)	25.6 MB

The evaluation loss is *lower* than the training loss. This is consistent with the absence of overfitting after a single epoch, but it can equally be explained by several non-learning factors: the training loss is averaged over steps whose early values inflate the mean (the model starts untrained and the loss is highest on the first minibatches), LoRA dropout is active during training but not during evaluation, and the evaluation split may happen to be slightly easier than the training split. We therefore report the figure as an observation that does *not* indicate overfitting, not as evidence of superior generalisation. A longer run (2–3 epochs) and a held-out test-loss measurement would be needed to draw any stronger conclusion.

6.2 Qualitative Inspection on Three Test Samples

Only three test samples were examined manually in this first run; the observations below are therefore anecdotal and in no way constitute a quantitative evaluation. A proper evaluation on the full 412-record test split, using scanner-based metrics, is part of the next iteration (Chapter 8).

Table 6.2 summarises the three samples processed by Cell 5. For each sample we compare the human-committed fix with the model's output relative to the declared smell.

Table 6.2: Qualitative behaviour on three held-out test samples.

#	Declared smells	Human fix (ground truth)	Fine-tuned model output
1	`apt_no_install_recommends`, `unpinned_base_image` (Docker)	Removes `ATMOS_VERSION 1.210.0` → `1.214.0` — does not address the declared smell	Adds `-allow-downgrades` on apt, introduces `addgroup/adduser` and `USER appuser` — an independent, sensible hardening
2	`uri_credentials` (Terraform)	Changes `recovery_window_in_days` `0` → `7` — unrelated to URI credentials	Regenerates the block with additional IRSA / IAM policy scaffolding, keeps `0` in place, does not target URI credentials
3	`unpinned_base_image` (Docker)	Pins a build-stage Go image by `sha256:...` digest (a real pin)	Does not pin any base image; performs unrelated hardening (`apt-get upgrade`, LABELS, no-install-recommends)

With the strong caveat that $n = 3$, the following observations are noted for follow-up:

- **Syntactic validity**: on these three outputs the model produced well-formed Dockerfiles and Terraform files — no broken `FROM`/`RUN` sequences and no malformed HCL. Whether this holds in general is unknown.
- **Domain awareness**: the outputs contain idiomatic hardening patterns (non-root user creation, label metadata, `-no-install-recommends`, AWS IRSA trust policies). Such patterns are plausibly learned from the training data, but they could also originate in the base model's pretraining corpus; this experiment does not distinguish between the two sources.
- **Alignment with declared smells**: on two of the three samples the model's fix did not address the labelled smell. The human ground truth also failed to address

the declared smell on those same two samples, which suggests the issue lies partly in the labels themselves rather than only in the model.

This last observation is critical. Commit-scraped datasets inherit the label noise of commit messages: a commit titled "fix: security" frequently bundles unrelated refactors, and the regex-based smell classifier has no reliable way to isolate the security-relevant part of the diff. Scanner validation partially mitigates this problem, but only at the *record* level, not at the *hunk* level. We return to this in Chapter 7.

6.3 Adapter Artefacts

The Kaggle output directory `/kaggle/working/lora_out` contains everything required for downstream use:

- `adapter_model.safetensors`, 25.6 MB — the trained LoRA weights.
- `adapter_config.json` — the Parameter-Efficient Fine-Tuning (PEFT) configuration (rank, target modules).
- `tokenizer.json`, `tokenizer.model`, `tokenizer_config.json`, `special_tokens_map.json` — the Gemma-2 tokenizer.
- `training_args.json` — a dump of the command-line parameters for provenance.
- Two intermediate checkpoints (steps 400 and 403) kept by the trainer for safety and discarded after download.

The full directory is downloaded to the laptop and copied to `Code/training/lora_out/` for subsequent local integration.

Chapter 7

Discussion and Limitations

7.1 Interpretation of Loss Values

Absolute loss values are not directly comparable across papers: the target length distribution and the tokeniser both affect the numeric value. We therefore restrict ourselves to a few narrow observations about the relative behaviour of this single run:

- The reported training loss goes from 0.745 on the 200-sample smoke test to 0.528 on the full corpus. Both figures are the *average over steps* within their respective run; they are not directly comparable because a 25-step run is dominated by early high-loss steps while a 403-step run is not. The comparison should be read as a sanity check that more data does not make things worse, not as a learning curve.
- Evaluation loss is below training loss, which — as discussed above — is consistent with no overfitting but is not, on its own, evidence of strong generalisation.
- We do not know how close this run is to the label-noise floor. If a substantial share of the ground-truth fixes do not actually address the declared smell (Section 7.2), no training signal can push the loss below that irreducible noise, and the current loss values may already be close to what is achievable with this corpus.

7.2 Label Noise as the Binding Constraint

The three-sample inspection is too small to quantify anything, but it is consistent with a limitation that is visible by construction in the dataset. Commit-scraped before/after pairs are aggregated at the granularity of a *file*, whereas a single commit may contain several unrelated modifications. The regex-based classifier infers the smell type from keywords in the commit message; the scanner validation pass confirms only that the file *before* was vulnerable according to at least one scanner, not that the removed lines in the diff actually address that specific vulnerability.

The practical consequence is that the training signal is an average of (a) genuine (insecure, fixed) pairs and (b) pairs where the recorded fix is unrelated to the declared

smell. A model trained on such a mixture is likely to learn a distribution of "things that look like fix commits" rather than a precise "smell → patch" mapping. The magnitude of this effect has not been measured; doing so is part of the next iteration.

Two remediations are plausible and will be tested in subsequent runs:

1. **Hunk-level filtering**: rerun the preparation script with an extra filter that requires the *diff itself* to contain a regex match for the declared smell pattern (e.g., for `hardcoded_credential`, the removed lines must contain a credential-like literal). The expected trade-off is a smaller but cleaner training set; the actual sizes are to be measured.

2. **Use of scanner test fixtures**: Checkov and KICS ship paired "failed" and "passed" example files. Using them as a high-purity subset — possibly in combination with the scraped data — would give a corpus whose labels are, by construction, what the scanners themselves check.

7.3 Experimental Limitations

This is a first exploratory run and most of its choices were made for tractability rather than optimality. The main limitations are:

- **Small model**: the base is only 2 B parameters. Larger open-weight models (3 B, 7 B, 9 B) are plausibly stronger on this task; none has been tried here.

- **Subset of the data**: we trained on 3,227 records, a subset selected for length and non-noop changes. The larger (noisier) pool of up to ≈ 10,791 scanner-validated records has not been used.

- **Single epoch**: one epoch is convenient for a first attempt but commonly underfits. Two to three epochs with early stopping are a more standard setting.

- **No hyperparameter search**: learning rate, LoRA rank, and target modules were copied from published defaults. A small grid search was not performed.

- **Single random seed**: the whole run was carried out once, with seed 42. Run-to-run variance has not been measured.

- **Single prompt template**: only one user-turn phrasing was used. Prompt diversity is a common robustness lever we did not exercise.

- **No downstream metric**: the loss figures reported here measure token-level agreement with the ground truth, not patch validity. The metric of interest — whether the generated patch causes the scanners to emit zero warnings on the fixed file — can only be computed inside the full agentic pipeline and has not been computed.
- **Qualitative inspection of** $n = 3$: any statement about output quality is anecdotal at this stage.

7.4 Threats to Validity

Given the preliminary nature of the run, the threats to validity are substantial:

- *External validity*: the dataset is drawn from public repositories and is skewed toward Docker and Kubernetes files. Behaviour on private-cloud Terraform modules or on unseen smell types is unknown.
- *Construct validity*: "security smell" is operationalised as "a pattern detected by Checkov, tfsec, or KICS". This definition excludes business-logic vulnerabilities and is at best a proxy for real-world security risk.
- *Internal validity*: a single training run was conducted with one random seed, one prompt template, and a fixed hyperparameter choice. None of these factors has been varied, so the reported numbers cannot be accompanied by any measure of uncertainty.

Chapter 8

Next Steps and Integration Path

8.1 From LoRA Adapter to Deployable Ollama Model

The trained artefact is a 25 MB LoRA adapter, not a runnable model. Three steps are required to make it available inside the agentic framework.

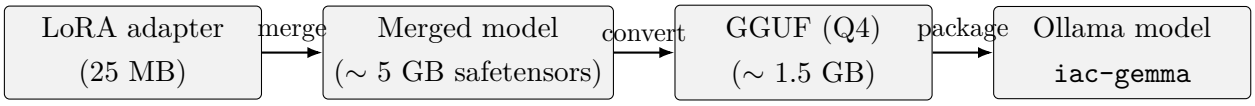

Figure 8.1: From adapter to deployable Ollama model.

Step 1 — Merge. A short script using `PeftModel.merge_and_unload()` fuses the adapter back into the base weights, producing a self-contained model directory. Storage cost rises to roughly 5 GB (the size of the full Gemma-2-2B) but this file never ships: it is an intermediate artefact.

Step 2 — Convert to GGUF. The `llama.cpp/convert_hf_to_gguf.py` utility translates the merged HF directory into a single GGUF file. A subsequent `llama-quantize` pass reduces it to 4-bit precision (Q4_K_M) and brings the file size down to roughly 1.5 GB. This is what gets distributed and loaded by Ollama.

Step 3 — Package for Ollama. A minimal `Modelfile` declaring the base GGUF, the Gemma-2 chat template, and a security-focused system prompt is passed to `ollama create iac-gemma`. Thereafter the model can be invoked identically to any other Ollama model.

On paper, the only change required on the framework side is to set `OLLAMA_MODEL=iac-gemma` in the `.env` file: the existing `FixGenerator` class (in `Code/src/generator/fix_generator.py`) discovers the backend and model through environment variables. This integration has not yet been exercised end to end and may require adjustments to the chat template or stop tokens.

8.2 Config B Evaluation

Once the model is live in Ollama, the framework's evaluation script `Code/scripts/evaluate.py` can be re-run on the eight-file labelled dataset to produce *Config B* numbers:

- **Config A** (reported in Rapport 2, Chapter 6): Checkov alone. F1 = 0.132, Precision = 0.084, Recall = 0.304.
- **Config B** (this report, pending): contextual analyzer + fine-tuned LLM + external validator (Checkov), full agentic pipeline.
- **Config C** (deferred): same as Config B with self-consistency (N=5 samples) and iterative retrieval refinement.

The primary metrics of interest for Config B, consistent with Rapport 2, are Patch Validity Rate, Smell Elimination Rate, No-New-Issues Rate, and a macro-averaged F1 over the ground-truth smell set.

8.3 Dataset Quality Improvements

Regardless of the evaluation outcome, the label-noise analysis of Chapter 7 motivates an incremental improvement of the dataset preparation pipeline:

1. Add a `-diff-matches-smell` filter in `01_prepare_dataset.py` that keeps only records whose diff contains a regex match for the declared smell.
2. Introduce a separate high-purity corpus consisting only of the Checkov and KICS test fixtures (1,850 records). Evaluate training on this small clean corpus against the larger noisy one.
3. Extend version 2 of the scraper (currently incomplete, introduces the GHArchive discovery path) to include a diff-hunk-level classifier rather than only a file-level one.

8.4 Model Scaling

If the Config B measurement, once available, proves insufficient, two axes of scaling are open:

- **Larger base model**. Qwen2.5-Coder-3B-Instruct is listed as a drop-in alternative in our training script; it is non-gated and advertised as a code-specialised model. Fine-tuning it is, in principle, compatible with Kaggle's T4 memory budget, but this has not been verified.
- **Longer training**. Two-to-three epochs on the same corpus, with early stopping on validation loss. Additional GPU-time cost is roughly proportional to the number of epochs.

8.5 Long-Term Outlook

The fine-tuning effort described in this report is one building block in a larger research programme. In the full vision the framework iterates between RAG-augmented generation, scanner-based validation, and selective retraining on the examples the validator marks as failing. The current experiment validates that the generator module can be specialised cheaply and deployed locally; it does not by itself close the loop. Subsequent reports will address self-training from validator feedback and the introduction of hunk-level label distillation.

Chapter 9

Conclusion

This report documents a first, preliminary fine-tuning experiment carried out within the agentic IaC security framework proposed in Rapport 2. From a 31,748-record scanner-validated scrape we extracted a 4,050-record subset; we then ran one pass of QLoRA fine-tuning on `google/gemma-2-2b-it` — a small 2 B-parameter base model — on Kaggle's free T4 x2 tier (a single device was used by the training script), observing a training loss of 0.528 and an evaluation loss of 0.467 in 1 h 47 min. A 25 MB adapter was produced and manually inspected on three test samples.

The main value of this iteration is methodological: the end-to-end pipeline (scrape → filter → train → adapter) runs within the constraints of a free GPU tier, and the dominant obstacle to better results has been identified as label noise in the commit-scraped ground truth rather than a property of the model or the training procedure. No claim of competitive performance is made at this stage.

The immediate next steps are concrete but unexecuted: merge the adapter, convert it to GGUF, serve it through Ollama, and run the framework's evaluation script to produce the first *measured* Config B numbers. In parallel, a stricter hunk-level label filter and an experiment on a scanner-fixture-only subset are planned to test the label-noise hypothesis directly. Only after these steps will it be possible to populate Chapter 6 of the final thesis report with a meaningful Config A/B/C comparison.